

# Interpretable Control for Unstable Systems via SINDy-RL

Patrick Smith · Andrew Falcone

ME 595 · Spring 2026

## 1 Introduction

### 1.1 The Deployment Gap

The proliferation of reinforcement learning in autonomous systems has produced controllers of remarkable capability, but capability alone is insufficient for safety-critical deployment. Regulators increasingly require that algorithmic decisions be explainable and auditable [8], and recent frameworks such as the EU AI Act impose formal interpretability requirements on high-risk applications [8]. A nine-thousand-parameter multilayer perceptron offers no handle for stability proofs or formal verification, and its memory and compute footprint makes it unsuitable for microcontroller execution. If the controller is instead a closed-form polynomial equation, each term can be audited by an engineer, bounding arguments can be constructed analytically, and the policy fits in kilobytes with evaluation requiring only a dot product. This representational gap between what deep RL produces and what deployed systems can accept motivates a growing body of work on inherently interpretable models [9] — controllers whose structure is transparent by construction, not explained after the fact.

### 1.2 SINDy and the Data Problem for Unstable Systems

Sparse Identification of Nonlinear Dynamics (SINDy) [2] offers a principled path to interpretable governing equations: given a library of candidate functions over the state and input, sparse regression identifies which terms actually drive the dynamics, discarding the rest. The resulting model is compact, physically grounded, and composed of a small number of terms that a practitioner can read and reason about. The difficulty is data. SINDy requires state-transition pairs that adequately cover the relevant region of the state space. For a stable system, a random policy will visit that region naturally. For an unstable equilibrium — such as an inverted pendulum — a random policy crashes in a handful of steps, and the near-upright transitions that SINDy needs to identify the governing dynamics are entirely unvisited. The system cannot provide the training data without a controller it does not yet have.

The resolution is to co-train the dynamics model and the controller iteratively. Sutton’s Dyna architecture [6] alternates between training a policy in a learned model and collecting new data from the real environment, so each iteration improves both components. Zolman et al. [1] apply this principle to SINDy by using an ensemble SINDy model as the Dyna surrogate, yielding a framework — SINDy-RL — that bootstraps the data problem while retaining interpretability as a downstream option.

### 1.3 This Work

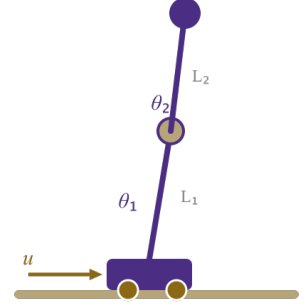
We implement Algorithm 1 from Zolman et al. [1] on the inverted double pendulum (IDP), a demanding two-link benchmark with two coupled unstable modes and a narrow region of attraction. The implementation required resolving a series of non-obvious engineering obstacles — a polynomial degree ceiling, a silent filter geometry bug, and surrogate exploitation — each of which prevented convergence on early attempts and each of which is described in detail in §4.3. The pipeline delivers two outputs: (1) a data-efficient neural network policy trained via the SINDy-RL Dyna loop, using  $14.5\times$  fewer real-environment interactions than a full-order PPO baseline; and (2) a degree-3 polynomial distilled from that policy, which achieves 65% task

success while remaining a printable, closed-form expression with 165 terms — 59× more compact than the baseline network.

## 2 Technical Background

### 2.1 The Testbed

The inverted double pendulum on a cart (InvertedDoublePendulum-v5, MuJoCo/Gymnasium) consists of two rigid links of equal length  $L_1 = L_2 = 0.6$  m, connected sequentially and mounted on a sliding cart. The physical state is six-dimensional:  $\mathbf{x} = [x_{\text{cart}}, \theta_1, \theta_2, \dot{x}, \dot{\theta}_1, \dot{\theta}_2]$ , where  $\theta_1$  and  $\theta_2$  are the joint angles measured from vertical. The observation presented to the controller is nine-dimensional, replacing raw angles with their sine and cosine encodings to avoid angle-wrapping discontinuities. The single control input is a horizontal force applied to the cart.



**Figure 1.** IDP geometry. State  $\mathbf{x} = [x, \theta_1, \theta_2, \dot{x}, \dot{\theta}_1, \dot{\theta}_2]$ . Tip height  $h = L_1 \cos \theta_1 + L_2 \cos(\theta_1 + \theta_2) \in [0, 1.2]$  m; episode ends at  $h \leq 1.0$  m.

The tip height  $h = L_1 \cos \theta_1 + L_2 \cos(\theta_1 + \theta_2)$  ranges from 0 (fully collapsed) to 1.2 m (both poles vertical). The episode terminates when  $h \leq 1.0$  m, leaving a near-upright band of only 0.2 m separating success from failure. The reward at each timestep is:

$$r_k = 10 \cdot \mathbf{1} - (h_k - 2)^2 - 0.01 x_{\text{tip}}^2 - \varepsilon \|\dot{\theta}\|^2$$

where the alive bonus ( $\approx 10/\text{step}$ ) dominates when the system remains upright and the term  $(h - 2)^2$  approaches zero as  $h \rightarrow 1.2$  m. Episodes last at most 1,000 steps (50 seconds at  $\Delta t = 0.05$  s).

### 2.2 SINDy-C: Sparse Dynamics Identification with Control

SINDy [2] regresses the state-transition residual  $\mathbf{x}_{k+1} - \mathbf{x}_k$  against a library of candidate functions evaluated at each state-action pair:

$$\mathbf{x}_{k+1} - \mathbf{x}_k = \underbrace{\Theta(\mathbf{x}_k, u_k)}_{\text{library}} \cdot \underbrace{\Xi}_{\text{sparse coefficients}}$$

For control-affine systems (SINDy-C [3]), the input  $u_k$  is included as an additional library variable. The Sequentially Thresholded Least Squares (STLSQ) algorithm iteratively zeros out coefficients whose magnitude falls below a threshold  $\lambda$ , promoting sparsity in  $\Xi$ . The degree of the polynomial library is a design parameter: a degree- $d$  library over  $n$  variables contains  $\binom{n+d}{d}$  candidate terms. For the IDP with a 7-dimensional augmented state-action vector, degree-2 yields 36 features and degree-3 yields 120 — a distinction that proved critical to convergence (§4.3).

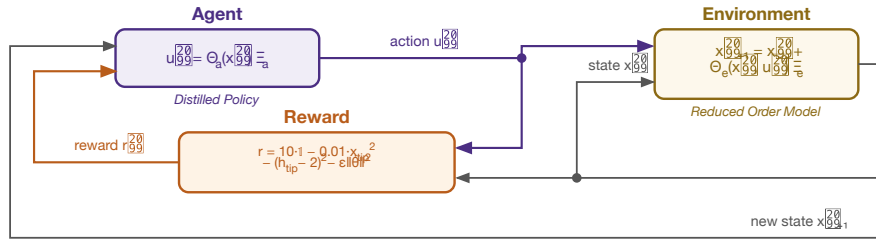
### 2.3 E-SINDy: Ensemble Uncertainty Quantification

A single SINDy model provides a point estimate of the dynamics with no uncertainty information. Fasel et al. [4] address this with Ensemble SINDy (E-SINDy): fit  $M$  independent SINDy models on 80% bootstrap subsamples of the data, then at inference time report the mean and standard deviation of predictions across the ensemble. For the IDP we use  $M = 10$  models. At each surrogate step,  $\text{predict}(\mathbf{x}, \mathbf{u})$  returns  $(\mu_\Delta, \sigma_\Delta)$ ,

where  $\mu_\Delta = \frac{1}{M} \sum_m \hat{\mathbf{x}}_{k+1}^{(m)} - \mathbf{x}_k$  and  $\sigma_\Delta$  is the per-component standard deviation across ensemble members. High  $\sigma_\Delta$  indicates a state-action region where the ensemble members disagree — a signal that the surrogate is extrapolating beyond its training distribution. Following Zolman et al. §3.5 [1], we convert this diagnostic into an active penalty: the RL reward inside the surrogate is reduced by  $\kappa \cdot \text{mean}(\sigma_\Delta)$  per step (with  $\kappa = 5.0$ ), steering policy optimization away from high-uncertainty states.

## 2.4 Dyna-Style MBRL and Behavioral Cloning

The Dyna architecture [6] interleaves model-based planning (cheap, unlimited rollouts inside the learned surrogate) with real-environment data collection (expensive, limited). In SINDy-RL [1], the surrogate is the E-SINDy model, and the planner is Proximal Policy Optimization (PPO [5]). Figure 2 shows the standard RL control loop; within SINDy-RL, the “Environment” box is instantiated twice — once as the E-SINDy surrogate during policy training (many cheap rollouts) and once as real MuJoCo during data collection (a small number of expensive rollouts per iteration).



**Figure 2.** The reinforcement learning control loop. In the SINDy-RL Dyna framework, the environment box is used in two modes: as the inexpensive E-SINDy polynomial surrogate for policy training, and as the full MuJoCo simulator for real data collection and evaluation. The agent learns from both.

The algorithm proceeds as follows. An initial controller (Schroeder multi-sine sweep [10], a deterministic band-limited excitation sequence that minimizes crest factor to avoid premature falls) collects a bootstrap dataset  $\mathcal{D}$  from the real environment. Then for each Dyna iteration: the E-SINDy ensemble is refit on near-upright transitions filtered from  $\mathcal{D}$ ; PPO is run for 100k steps inside the surrogate, warm-starting from the previous iteration’s weights; the resulting policy is deployed in real MuJoCo to collect 4,000 new transitions, which are appended to  $\mathcal{D}$ . After the loop converges, the best checkpoint by real episode length is optionally distilled into a sparse polynomial via behavioral cloning:

$$\min_{\Xi} \|\Theta_{\text{obs}}(X) \Xi - U^*\|_2 \quad (\text{STLSQ, threshold } \lambda = 0.05)$$

where  $X$  is a matrix of 8-dimensional observations,  $U^*$  are the corresponding NN policy actions,  $\Theta_{\text{obs}}$  is the degree-3 polynomial library on observations, and  $\Xi$  is the distilled coefficient vector. Perturbation augmentation [7] — adding Gaussian noise to each expert state and re-querying the NN oracle — expands the dataset 5× to reduce distribution shift without additional MuJoCo rollouts.

### 3 Methods

#### 3.1 Baseline: Full-Order PPO

The performance ceiling is a standard PPO agent trained with unlimited real-environment access. We use eight parallel environments, a two-hidden-layer [64, 64] MLP with tanh activations (9,731 parameters), and train for 400,000 total timesteps, saving the best checkpoint by mean evaluation reward (evaluated every 3,125 steps over 20 episodes). This policy is a reference point only — it is not used as a distillation teacher. The SINDy-RL distillation teacher is the best Dyna-loop checkpoint, trained on a small fraction of the data.

#### 3.2 SINDy-RL Pipeline

The SINDy-RL pipeline implemented in `notebooks/sindy-rl.ipynb` comprises five stages executed from a single notebook. In the bootstrap stage, 300 episodes of Schroeder multi-sine excitation collect 2,897 state-transition pairs. The E-SINDy fit stage filters these transitions to near-upright instances (tip height  $> 1.10$  m, corresponding to poles within  $\approx 24^\circ$  of vertical) and fits 10 degree-3 SINDy-C models on 80% subsamples, stacking the resulting coefficient matrices into a `FastEnsemblePredictor` that evaluates the full ensemble in a single matrix multiply. The surrogate PPO stage wraps the predictor in an `EnsembleSurrogateEnv` that applies the uncertainty penalty and terminates episodes when the simulated pole falls below 1.0 m; training runs for 100k steps with early stopping if mean surrogate episode length remains below 5 steps after 50k steps (indicating a non-functional surrogate). Real data collection deploys the policy in MuJoCo for 4,000 steps. The loop repeats, warm-starting PPO from the previous checkpoint; if exploitation is detected (surrogate reward  $> 3 \times$  previous AND real episode length  $< 50\%$  of best seen), the next iteration warm-starts from the best-real-env checkpoint rather than the exploited policy.

After the Dyna loop, distillation collects 50k expert transitions by rolling out the best Dyna checkpoint in MuJoCo. Each transition is augmented  $5\times$  with Gaussian noise on the state (per-dimension  $\sigma$ : [0.02, 0.02, 0.02, 0.05, 0.10, 0.10]) and re-queried from the NN oracle, expanding the training set to 300k observation-action pairs. A degree-3 polynomial is then fit via STLSQ with threshold 0.05 on the 8-dimensional observation (sin/cos-encoded).

#### 3.3 Metrics

Two goals drive evaluation: data efficiency and interpretability. Data efficiency is measured by real-environment step count — every step costs real hardware time and wear in physical deployment, so fewer is unconditionally better. Task performance is measured by success rate ( $\geq 500$  steps, equivalent to 25 seconds of balance) and mean episode length; the latter provides a graded signal that distinguishes a policy that balances for 600 steps from one that falls after 60. Surrogate quality is tracked by SINDy RMSE on held-out near-upright transitions, serving as a proxy for how much the Dyna loop can trust its internal model. Distillation quality is measured by OLS  $R^2$  and STLSQ term count, which together characterize how faithfully the polynomial captures the NN’s decisions and how compact the result is.

### 4 Preliminary Results

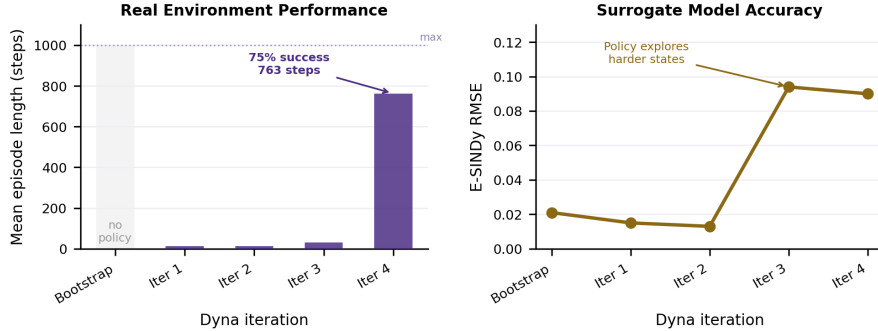
#### 4.1 Baseline

The full-order PPO baseline achieves mean reward  $9,324 \pm 2$ , 100% success rate, and mean episode length of 1,000/1,000 steps — a perfect score on the task. This performance costs 400,000 real simulator interactions

and produces a 9,731-parameter network that offers no interpretability, no analytic stability guarantees, and no path to embedded deployment. It serves as the performance ceiling against which the data-efficient SINDy-RL approach is measured.

## 4.2 Dyna Loop Convergence

The Dyna loop converged in four iterations, accumulating 27,512 real-environment steps — **14.5× fewer than the baseline**. Figure 3 shows the convergence trajectory: mean real episode length (bars) and SINDy RMSE (line) across iterations.



**Figure 3.** SINDy-RL Dyna loop convergence. Bars (left axis): mean real episode length per iteration. Line with markers (right axis): SINDy RMSE on near-upright transitions. The RMSE rise at iterations 3–4 reflects the improving policy visiting states further from vertical, where polynomial dynamics are less accurate; the surrogate remained sufficiently accurate in the near-upright band to train a transferable policy. Iteration 4 achieved 80% success and mean length 805 steps.

| Iteration | Cumulative real steps | SINDy RMSE | Surrogate mean len | Real mean len | Success    |
|-----------|-----------------------|------------|--------------------|---------------|------------|
| Bootstrap | 2,897                 | 0.021      | —                  | —             | —          |
| 1         | 7,023                 | 0.015      | 12.6               | \$≈\$12       | 0%         |
| 2         | 11,150                | 0.013      | 12.7               | \$≈\$12       | 0%         |
| 3         | 15,461                | 0.094      | 31.1               | \$≈\$31       | 0%         |
| 4         | 27,512                | 0.090      | <b>805</b>         | <b>805</b>    | <b>80%</b> |

*†Episode lengths are within-loop real-environment evaluations (20 episodes per iteration). A dedicated post-loop evaluation of the iteration-4 checkpoint over a separate 20-episode trial gave mean length 763 steps and 75% success; these are the definitive figures used in §4.4 and §5.*

The RMSE rose from 0.013 to 0.094 at iteration 3 and remained elevated at iteration 4. This is not a sign of model degradation but a natural consequence of a better policy exploring states further from the upright equilibrium, where the degree-3 polynomial is less accurate. The surrogate nevertheless remained sufficiently accurate in the near-upright band — the region that matters for policy training — to produce a policy that transferred to real MuJoCo at iteration 4.

### 4.3 Obstacles on the Path to Convergence

Three non-obvious implementation obstacles prevented convergence on early attempts. Each was diagnosed from observable signals and fixed with targeted interventions.

**Degree-2 RMSE ceiling.** An initial implementation using a degree-2 polynomial library (36 features) was run for 25 Dyna iterations with data accumulating from 5,825 to over 90,000 transitions. RMSE oscillated between 0.10 and 0.16 regardless of data volume; real episode lengths grew from 6 to only 22 steps over the full 25 iterations — effectively stalled. When RMSE fails to decrease with  $10\times$  more data, the cause is a model capacity ceiling rather than a data quantity problem. A degree-2 library cannot express the cross-coupling between the two angular modes that dominates IDP dynamics, because the leading nonlinear interaction terms are cubic (e.g.,  $\cos \theta_1 \cdot \cos \theta_2 \cdot \dot{\theta}_1$ ). Zolman et al. [1] use degree-2 for all benchmark environments, but those environments have single unstable modes with weaker inter-state coupling. The fix — `SINDY_DEGREE = 3`, expanding the library from 36 to 120 features — reduced RMSE to 0.013 within two iterations.

**Near-upright filter geometry bug.** The Dyna loop is intended to filter the accumulated dataset  $\mathcal{D}$  to near-upright transitions before refitting E-SINDy, ensuring the surrogate is accurate precisely where the policy needs it. This filter retains only transitions where tip height  $h > \text{SINDY\_H\_MIN}$ . With `SINDY_H_MIN = 1.6` m and a physical maximum tip height of  $L_1 + L_2 = 0.6 + 0.6 = 1.2$  m, no transition could ever pass the filter. The fallback logic (use all data when the filtered count is below 50) fired silently every iteration, making the filter a complete no-op.

The root cause was a semantic error in a named constant. Because the E-SINDy surrogate computes rewards analytically from state — MuJoCo is not running inside the surrogate — we implemented our own `reward_done()` function replicating Gymnasium’s formula  $r = 10 - (h - 2)^2 - \dots$ , and introduced a named constant `TIP_HEIGHT_TARGET = 2.0` to match Gymnasium’s hardcoded literal. That constant is correct for the reward function — our reward matches Gymnasium’s to machine precision — but our mistake was then using `TIP_HEIGHT_TARGET` as the reference for the filter threshold, setting `SINDY_H_MIN`  $\approx 0.8 \times 2.0 = 1.6$ . The value 2.0 is a reward-shaping offset, not the physical height the pendulum can reach. The correct reference is segment geometry: maximum achievable  $h = 1.2$  m. The fix — `SINDY_H_MIN = 1.10` m, derived from segment lengths and placing the threshold at poles within  $\approx 24^\circ$  of vertical — activated the filter and allowed E-SINDy to focus on the dynamically relevant region.

**Surrogate exploitation.** A diagnostic 30-iteration run (conducted before the safeguards described below were in place) revealed a catastrophic failure mode: at iteration 6, surrogate reward jumped from 497 to 4,525 (a  $9\times$  increase) while real episode length collapsed from 414 to 56 steps. The policy had found action sequences that the degree-3 polynomial predicted as highly rewarding but that did not correspond to real physics. When warm-started into iteration 7, the exploited policy collected mostly falling trajectories from real MuJoCo, degrading the SINDy fit, which worsened the surrogate for iteration 8, compounding the failure across all subsequent iterations.

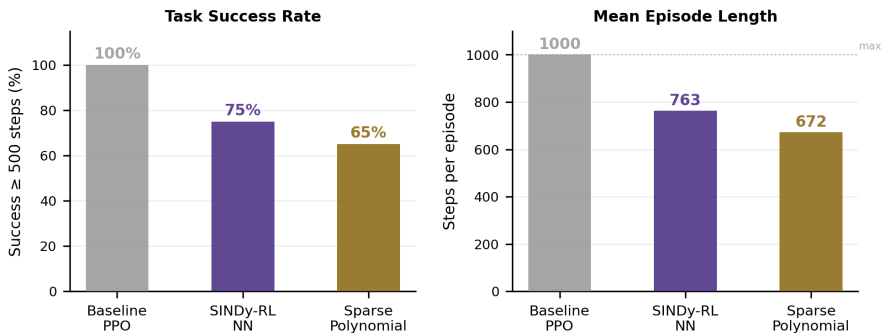
Uncertainty penalization alone is insufficient to prevent this failure. All 10 ensemble members share the same polynomial basis; in extrapolated regions, they all make the same wrong prediction simultaneously, so ensemble disagreement  $\sigma_\Delta$  is low even though the shared prediction is wrong. Rollback alone is also insufficient: it detects exploitation after the fact but does not prevent the exploited iteration from collecting low-quality data and degrading the fit. Both mechanisms are needed together. During surrogate PPO, the reward penalty `reward -= 5.0 * mean(sigma_delta)` steers optimization away from states where the ensemble disagrees. After each real evaluation, if surrogate reward exceeds  $3\times$  the previous iteration’s value

and real episode length is below 50% of the best seen, the next iteration warm-starts from the best-real-env checkpoint rather than the exploited policy. The final convergence run (§4.2) never triggered exploitation because both safeguards were active from the start, consistent with Zolman et al.’s §3.5 [1] recommendation to treat ensemble variance as an active penalty rather than a passive diagnostic.

#### 4.4 Policy Distillation

Behavioral cloning from the best Dyna checkpoint — the iteration-4 policy with a real-environment mean length of 805 steps — produced a degree-3 polynomial achieving **65% success and mean episode length 672** on 20 evaluation episodes in real MuJoCo (Figure 4). The distillation required resolving three additional obstacles not described in Zolman et al. [1].

Using the final loop policy (iteration 4 end-state after 100k surrogate steps) as the teacher rather than the best checkpoint yielded a distilled polynomial with mean length 32.6 steps and 0% success; the policy at the end of surrogate training is not necessarily the best policy, as unconstrained surrogate optimization may have drifted from the real-env peak. Using the best-checkpoint teacher but a degree-2 polynomial library gave OLS  $R^2 \approx 0.905$  regardless of data volume — the same capacity ceiling encountered in the dynamics model. Switching to degree-3 (165 candidate features on the 8-dimensional observation space) raised  $R^2$  to 0.9916. Finally, distribution shift — the polynomial is only trained on states the NN naturally visits, but deployment pushes the system off that manifold — required perturbation augmentation [7]:  $5\times$  noise-augmented re-queries of the NN oracle expanded the 50k-transition dataset to 300k pairs, raising success from near-zero to 65% without additional MuJoCo rollouts.



**Figure 4.** Method comparison. Success rate and mean episode length for three approaches: baseline PPO (400k real steps, 9,731 parameters), SINDy-RL neural network (27,512 real Dyna steps, same architecture), and SINDy-RL sparse polynomial (77,512 total steps, 165-term degree-3 polynomial). The polynomial achieves 65% success at  $5.2\times$  lower data cost than the baseline.

**Sparsity results.** STLSQ with threshold 0.05 retained all 165/165 policy terms — no sparsity was achieved. The dynamics surrogate showed similarly dense structure: 690 nonzero coefficients out of 720 possible ( $120 \text{ features} \times 6 \text{ state dimensions}$ ). The IDP appears to require the full expressiveness of a degree-3 polynomial, with no terms that can be eliminated without degrading fit quality. This is in contrast to Zolman et al.’s results on lower-dimensional benchmarks, where STLSQ produced genuinely sparse models with  $\mathcal{O}(10)$  terms.

**Compactness and partial interpretability.** The 165-term polynomial is  $59\times$  more compact than the 9,731-parameter baseline network, fits in a few kilobytes, and can be evaluated without a matrix multiply stack. This is a meaningful practical gain, but it is not interpretability in the scientific sense of a short, auditable equation. Inspecting the dominant terms by coefficient magnitude reveals recognizable control structure: a

constant bias of  $-0.626$ ; proportional angle feedback on  $\cos \theta_1$  ( $+25.5$ ) and  $\cos \theta_2$  ( $-3.8$ ); velocity damping on  $\dot{x}$  ( $+4.4$ ),  $\dot{\theta}_1$  ( $+3.5$ ),  $\dot{\theta}_2$  ( $-1.1$ ); and a large inter-pole coupling term  $\cos \theta_1 \times \cos \theta_2$  ( $-158.2$ ) that is the physically expected dominant nonlinearity in a double pendulum. These dominant terms are analogous to a PD controller with coupling, which is physically sensible. However, the remaining  $\$120$  terms are small-coefficient cubic cross-products (e.g.,  $+0.0001 x^3$ ,  $+0.0003 x^2 \sin \theta_1$ ) with no obvious physical interpretation — they appear to encode the residual nonlinearity of the NN’s tanh activations rather than any mechanistic structure.

**Data efficiency and the distillation trade-off.** The clearest result of this work is data efficiency. The Dyna loop alone used 27,512 real-environment steps — **14.5× fewer than baseline** — to produce a NN policy with 75% success. The full pipeline including distillation used 77,512 steps — **5.2× fewer than baseline**. The 50k distillation steps are qualitatively cheaper than Dyna steps: the teacher policy is already competent (75% success), so those rollouts are mostly stable near-upright operation rather than costly exploration failures. Nevertheless, distillation costs 50k additional real interactions and *reduces* task performance from 75% to 65%. That trade-off is only justified by a hard downstream requirement for a closed-form controller — embedded hardware with no floating-point stack, formal stability verification, or regulatory auditability. If data collection is the binding constraint, the Dyna loop NN (27,512 steps, 75% success) is the stronger deliverable; the polynomial is an interpretability option, not a performance improvement.

## 4.5 Code Repository

All code and results are available at: [https://github.com/falconeaj1/ME\\_595](https://github.com/falconeaj1/ME_595)

Key notebooks: `notebooks/full-order-simulation.ipynb` (baseline PPO training and evaluation) and `notebooks/sindy-rl.ipynb` (SINDy-RL Dyna loop, E-SINDy surrogate, sparse policy distillation). Professor Michelle Hickner has been added as a collaborator (GitHub: mhickner).

## 5 Summary and Next Steps

This work implements SINDy-RL on the inverted double pendulum and achieves two deliverables: a data-efficient neural network policy trained in 27,512 real steps (14.5× more efficient than the baseline) that achieves 75% task success, and a 165-term degree-3 polynomial distilled from that policy that achieves 65% success using 77,512 total steps (5.2× more efficient). The polynomial is 59× more compact than the baseline network and exposes physically recognizable dominant terms — angle feedback, velocity damping, and inter-pole coupling — though STLSQ achieved no sparsity and the full 165-term expression cannot be audited in the way a five-term equation could. Three non-trivial implementation obstacles (degree-2 capacity ceiling, filter geometry bug, surrogate exploitation) had to be resolved before convergence; these engineering contributions extend Zolman et al.’s algorithm description with practical diagnostics and safeguards applicable to other unstable benchmark systems.

| Approach                | Real-env steps | Mean ep len | Success | Inspectable | Parameters |
|-------------------------|----------------|-------------|---------|-------------|------------|
| Baseline PPO            | 400,000        | 1,000       | 100%    | No          | 9,731      |
| SINDy-RL NN (best Dyna) | 27,512         | 763         | 75%     | No          | 9,731      |



| Approach                       | Real-env steps      | Mean ep len | Success | Inspectable          | Parameters |
|--------------------------------|---------------------|-------------|---------|----------------------|------------|
| SINDy-RL<br>Poly<br>(degree-3) | 77,512 <sup>†</sup> | 672         | 65%     | Partial <sup>‡</sup> | 165 terms  |

<sup>†</sup>27,512 Dyna real-env steps + 50,000 MuJoCo rollout steps for distillation data collection. The  $5\times$  perturbation augmentation re-queries the NN oracle on perturbed states — no additional MuJoCo interactions required.

<sup>‡</sup>All 165/165 terms retained (no sparsity). Dominant terms are physically recognizable; remaining approximately 120 small cubic cross-terms are not.

The immediate next step is closing the interpretability loop: rather than distilling a NN policy trained in a SINDy surrogate, train directly in a simplified or linearized SINDy surrogate so that the policy structure is constrained to match the model structure from the start. A second open question is the STLSQ threshold ablation: higher thresholds would produce sparser policies, but the performance-interpretability trade-off on the IDP has not been characterized.